# Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

**Thème principal :** Variables, Mémoire RAM, Affectation, Structures Conditionnelles et Structures Itératives

**Synthèse exécutive :** Ce cours pose les fondations indispensables du développement logiciel et de la pensée algorithmique en BUT Informatique. Il résout le problème de la traduction d'un processus logique du monde réel en une suite formelle, séquentielle et déterministe d'instructions exécutables par un ordinateur. La séance s'articule autour de trois piliers : la modélisation de la mémoire vive (RAM) à travers le quadruplet caractérisant une variable (Nom, Type, Valeur, Adresse) et la mécanique d'affectation destructive ; l'aiguillage du flux d'exécution séquentiel à l'aide de prédicats booléens et de structures conditionnelles (Si / Alors / Sinon) ; ainsi que l'automatisation de traitements répétitifs au moyen de structures itératives bornées (Pour) et non bornées (TantQue).

## Mémorisation et Manipulation des Données : Les Variables et la Mémoire RAM

En informatique, le concept de variable se distingue radicalement de sa définition mathématique statique. Il ne s'agit pas d'une inconnue abstraite dans une équation, mais d'une case mémoire physique concrète située dans la RAM de l'ordinateur. Une variable est obligatoirement caractérisée par un quadruplet : son Nom (qui permet au développeur de la manipuler), son Type (qui indique au système le nombre d'octets à réserver et le format de codage binaire), sa Valeur (le contenu actuel à un instant t) et son Adresse mémoire physique (emplacement hexadécimal en RAM). Lors de l'exécution d'une instruction d'affectation (ex: x ← x + 1), l'Unité Centrale de Traitement (CPU) applique un protocole en deux étapes chronologiques : premièrement, elle lit en mémoire RAM les valeurs des variables figurant à droite du symbole d'affectation et évalue l'expression globale ; deuxièmement, elle vient inscrire ce résultat final dans la case mémoire associée à la variable située à gauche. Ce processus est strictement destructif : le nouveau résultat écrase définitivement l'ancienne valeur. Le piège majeur en partiel consiste à utiliser une variable sans l'avoir initialisée. Si un algorithme cherche à lire une variable non initialisée dans un calcul, le système va récupérer les bits résiduels restés en mémoire vive (la 'valeur poubelle'), engendrant des calculs faux ou des comportements totalement imprévisibles du programme.

- Variable : Emplacement étiqueté réservé dans la mémoire vive (RAM) destiné à conserver une valeur modifiable pendant l'exécution du programme.
- Quadruplet d'une variable : Ensemble indissociable constitué de son Nom (identificateur), son Type (format et taille en octets), sa Valeur actuelle et son Adresse physique en RAM.
- Typage formel : Spécification explicite du domaine de valeurs (Entier, Réel, Chaîne, Booléen) qui détermine la quantité de mémoire allouée et les opérations autorisées.
- Affectation (← ou =) : Opération destructive d'écriture qui évalue l'expression située à droite et remplace irrémédiablement la valeur stockée à gauche.
- Valeur poubelle : Donnée binaire résiduelle non nettoyée présente en RAM, lue lorsqu'une variable est exploitée sans avoir été préalablement initialisée.

***Cas concret enseignant :*** *La métaphore de l'armoire d'archivage à tiroirs : Imaginez la mémoire RAM comme une grande armoire d'archivage contenant des tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon le type ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir précis, à jeter au recyclage le morceau de papier qui s'y trouvait auparavant (l'ancienne valeur), et à y déposer une nouvelle feuille portant l'inscription 150.*

## Prise de Décision : Les Structures Conditionnelles (Si / Alors / Sinon)

Par défaut, l'ordinateur exécute un programme de manière séquentielle, de la première à la dernière instruction, ligne après ligne. Pour doter l'algorithme d'un comportement adaptatif et intelligent, on introduit des structures conditionnelles. Lorsqu'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]' est rencontré, le processeur évalue

l'expression booléenne. Si le prédicat donne la valeur VRAI, l'ordinateur exécute exclusivement le [Bloc A], puis saute par-dessus le [Bloc B] pour reprendre le flux normal. Si la condition donne la valeur FAUX, le [Bloc A] est ignoré et seul le [Bloc B] (s'il existe) est exécuté. Pour formaliser des critères décisionnels complexes, la maîtrise des tables de vérité est requise : l'opérateur ET (AND) exige que l'ensemble des sous-conditions soient simultanément vraies pour valider le test global ; l'opérateur OU (OR) requiert qu'au moins une des sous-conditions soit vraie ; l'opérateur NON (NOT) inverse la valeur de vérité de l'expression.

- Structure d'Alternative (Si / Alors / Sinon) : Branchement logique détournant le flux d'exécution séquentiel selon la valeur de vérité d'une condition.
- Expression Booléenne (Prédicat) : Proposition logique évaluée par le processeur qui ne peut prendre que deux états strictement binaires : VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Connecteurs booléens permettant de composer des conditions complexes régies par leurs tables de vérité respectives.

***Cas concret enseignant :*** *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'expulser les billets de banque, l'automate évalue une condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale est VRAIE, le paiement est validé et les billets sont délivrés ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est restituée à l'usager.*

## Répétition et Automatisation : Les Structures Itératives (Boucles Pour et TantQue)

La véritable puissance des systèmes informatiques réside dans leur capacité à exécuter des séries d'instructions répétitives à très haute vitesse et sans altération. Deux structures itératives principales sont utilisées en algorithmique : 1. La boucle POUR (déterministe) : Elle est privilégiée lorsque le nombre d'itérations est connu avant d'entrer dans la boucle. Elle intègre nativement l'initialisation d'une variable compteur, le test de borne supérieure et l'incrémentation systématique à chaque itération (ex: traiter les N éléments d'un tableau). 2. La boucle TANT QUE (indéterminée) : Elle s'impose lorsque le nombre de répétitions dépend d'un événement dynamique au cours du programme (ex: valider la saisie d'un mot de passe utilisateur). La condition est évaluée AVANT chaque passage. Si elle s'avère vraie, le corps de la boucle s'exécute ; si elle est fausse au départ, le corps n'est jamais exécuté. La règle d'or d'une boucle TANT QUE est de modifier impérativement, à l'intérieur du corps de la boucle, au moins une variable engagée dans la condition de sortie. Si la condition reste indéfiniment VRAIE, l'algorithme tombe dans une boucle infinie qui bloque l'exécution et sature le processeur.

- Boucle Pour (FOR) : Structure itérative bornée et déterministe, utilisée lorsque le nombre précis de répétitions est connu à l'avance.
- Boucle TantQue (WHILE) : Structure itérative non bornée et dynamique, répétée tant qu'une condition logique demeure VRAIE.
- Variable de Contrôle & Incrémentation : Modification explicite d'un compteur au sein du corps de la boucle pour garantir le franchissement de la condition de sortie.
- Boucle Infinie : Dysfonctionnement critique lié à l'absence d'actualisation de la condition de sortie dans une boucle TantQue, gelant le programme et monopolisant les ressources CPU.

***Cas concret enseignant :*** *La métaphore de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est connu dès le départ et la fin est garantie après 10 tours). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la sortie dépend d'une observation météo réévaluée à chaque tour de piste).*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Pourquoi l'instruction 'x = x + 1' (ou 'x ← x + 1') est-elle valide en informatique alors qu'elle est mathématiquement impossible ?
**Réponse du professeur :** En mathématiques, le signe '=' exprime une égalité statique (l'équation x = x + 1 n'a aucune solution). En informatique, il s'agit d'une opération d'affectation dynamique séquentielle. Le processeur évalue d'abord la valeur à droite (la valeur actuelle de x augmentée de 1), puis il écrit ce nouveau résultat dans la case mémoire x à gauche, écrasant ainsi l'ancienne valeur.

**Question étudiant :** Que se passe-t-il concrètement si j'oublie de modifier la variable de contrôle à l'intérieur du corps d'une boucle TantQue ?
**Réponse du professeur :** Si la variable de contrôle n'est jamais modifiée dans le corps de la boucle, la condition de maintien reste constamment VRAIE. Le programme entre immédiatement dans une 'boucle infinie' : l'application gèle, ne rend plus la main et consomme 100% des ressources du processeur jusqu'à son interruption forcée.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire un programme C++ (respectant strictement la norme de l'IUT d'Anglet) qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

**Démarche de résolution :**
Étape 1 : Structuration du fichier source avec le cartouche d'en-tête et l'organisation en tripartition (// CONSTANTES, // VARIABLES, // TRAITEMENTS). Étape 2 : Déclaration des variables n, i et somme en tant qu'entiers (int). Étape 3 : Saisie sécurisée du paramètre n au clavier avec annotation du flux d'entrée // (clavier) >> saisir >> n. Étape 4 : Initialisation explicite de la variable 'somme' à 0 avant d'entrer dans la boucle afin d'écraser toute valeur poubelle en mémoire RAM. Étape 5 : Mise en place d'une boucle 'for' allant de i = 1 jusqu'à n avec incrémentation de 1 à chaque pas, appliquant la syntaxe d'indentation Allman. Étape 6 : À chaque itération, addition du compteur i à l'accumulateur 'somme'. Étape 7 : Affichage de la valeur finale calculée sur la sortie standard.

```
/*
  Programme : SommeDesEntiers
  But : Calculer et afficher la somme des entiers de 1 à N
  Date : 11/11/2026
  Auteur : Matéi
*/
#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int n;      // Limite supérieure saisie par l'utilisateur
    int i;      // Compteur / Variable de contrôle de boucle
    int somme;  // Accumulateur de la somme partielle

    // TRAITEMENTS
    cout << "Veuillez saisir un nombre entier positif N : ";
    // (clavier) >> saisir >> n
    cin >> n;

    // Initialisation stricte pour éliminer toute valeur poubelle en RAM
    somme = 0;

    // Calcul de la somme cumulative
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    cout << "La somme des entiers de 1 à " << n << " est égale à : " << somme << endl;

    return 0;
}
```

**Résultat : Pour une saisie N = 5, les itérations successives sont : somme = 0+1=1, puis 1+2=3, puis 3+3=6, puis 6+4=10, puis 10+5=15. Le programme affiche : 'La somme des entiers de 1 à 5 est égale à : 15'. Le résultat est déterministe et conforme aux exigences académiques.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence fondamentale de critère de choix entre une boucle POUR et une boucle TANT QUE ?**

*R : La boucle POUR s'utilise lorsque le nombre d'itérations est connu de façon déterministe avant d'entrer dans le traitement. La boucle TANT QUE s'impose lorsque le nombre d'itérations est indéterminé et dépend d'une condition dynamique réévaluée à chaque passage.*

**Q : Que valent les variables A et B après l'exécution exacte de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?**

*R : Après A ← 5 et B ← 3 : A=5, B=3. Après A ← A + B (5+3) : A=8, B=3. Après B ← A - B (8-3) : B=5. À la fin, A vaut 8 et B vaut 5 (cette séquence réalise l'échange des valeurs de deux variables).*

**Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) entraîne-t-elle la perte de points en partiel ?**

*R : Sans initialisation explicite, la variable conserve une 'valeur poubelle' (données binaire résiduelles en RAM). Effectuer une addition sur cette valeur fausse immédiatement tous les calculs ultérieurs du programme.*